NEXUS — Manual técnico y preparación para el examen H3

Programación II | Manual de estudio sin conexión | Documento en blanco y negro

Fuente analizada: código C# proporcionado por el estudiante, archivo de texto que contiene las clases del espacio de nombres NEXUS. En el repositorio, verificar que corresponda a Program.cs. No se revisaron archivos .csproj ni otros archivos del repositorio.

Este manual distingue los elementos realmente implementados de los ejemplos complementarios. Los resultados de ejecución se deducen del código; no se afirma haber ejecutado el proyecto.

I. Arquitectura y programación orientada a objetos

NEXUS es una aplicación de consola con un sistema de exploración y misión. Program.Main crea SistemaNexus e invoca Iniciar. El sistema crea un cadete, zonas, inventario y una misión. El menú permite explorar, consultar y usar objetos, ver objetivos y estado o salir.

Clases y responsabilidades (todas presentes en el código entregado)

• Program: punto de entrada Main; crea SistemaNexus.

• SistemaNexus: coordina menú, exploración, encuentros, recompensas y finalización.

• Personaje (abstracta): estado compartido Nombre y Vida, recibir daño y contrato abstracto Actuar.

• Cadete: personaje jugable; energía, experiencia e inventario.

• Robot y Enemigo: personajes especializados que actúan contra el cadete.

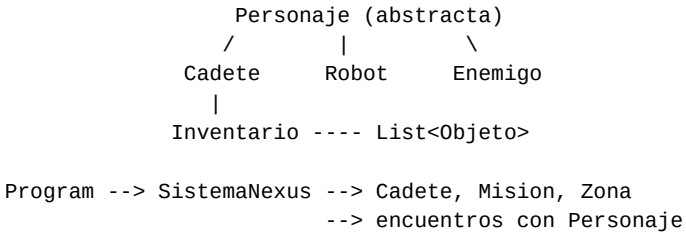
• Objeto: representa ítems con Nombre, Tipo y Poder.

• Inventario: almacena List<Objeto> y permite agregar, mostrar y utilizar.

• Zona: contiene nombre y descripción del lugar.

• Mision: guarda cuatro objetivos booleanos y comprueba si se completaron.

Diagrama de clases simplificado



La flecha desde Cadete a Inventario indica que Cadete contiene un inventario. Robot, Enemigo y Cadete heredan de Personaje. SistemaNexus mantiene referencias a Cadete, Mision y cuatro objetos Zona.

Programación estructurada frente a POO

En un diseño estructurado, las operaciones se organizarían principalmente en funciones y datos separados. En NEXUS, cada clase agrupa datos y comportamientos: Mision administra sus objetivos e Inventario administra sus objetos. La POO favorece responsabilidades separadas y reutilización.

II. Fundamentos de clases y objetos

Clase: molde que define datos y comportamientos. Objeto: instancia concreta. Ejemplo real: new Cadete(nombre) crea un cadete; new Robot("ROBOT R-07") crea un robot. Los atributos del estado se exponen mediante propiedades como Vida y Energia; el campo privado objetos guarda la lista del inventario.

```
Cadete cadete = new Cadete("Alex");
cadete.GastarEnergia(10);
// Energia comienza en 60 y queda en 50.
```

Métodos: GastarEnergia, RecuperarEnergia y GanarExperiencia modifican el estado. Constructores: Cadete(string nombre) llama a base(nombre, 100); Robot llama a base(nombre, 50) y Enemigo a base(nombre, 70). No hay sobrecarga de constructores en el código aportado.

Encapsulamiento: Energia tiene public get y private set, por lo que otras clases pueden leerla pero no asignarla directamente. Nombre y Vida tienen protected set, lo que permite cambios desde la clase base o derivadas. Inventario guarda su List<Objeto> en un campo private.

Uso real de base: en Cadete(string nombre) : base(nombre, 100) se inicializan las propiedades heredadas. No se observa uso de this. Ejemplo complementario de this y sobrecarga:

```
class EjemploCadete {
    public string Nombre { get; private set; }
    public EjemploCadete() : this("Sin nombre") { }
    public EjemploCadete(string nombre) { this.Nombre = nombre; }
}
```

III. Herencia, polimorfismo e interfaces

Personaje es la clase base abstracta. Cadete, Robot y Enemigo son derivadas. Heredan Nombre, Vida y RecibirDanio, pero especializan MostrarInformacion y Actuar mediante override. Actuar es abstract: cada derivada debe implementarlo. MostrarInformacion es virtual: permite reemplazar su comportamiento.

```
// Fragmento real de la arquitectura de NEXUS:
private void Encuentro(Personaje personaje) { /* ... */ }
// Robot y Enemigo pueden pasar como Personaje.
personaje.Actuar(cadete); // invoca la version del tipo real.
```

Polimorfismo: Encuentro(Personaje personaje) acepta tanto Robot como Enemigo. Si el objeto es Robot, Actuar quita 10 de vida y 5 de energía al cadete; si es Enemigo, quita 15 de vida. La versión ejecutada depende del objeto real, no solo del tipo de la variable.

Herencia significa ser un tipo de otro (Robot es Personaje). Composición significa contener o administrar otros objetos (Cadete contiene Inventario). Implementar una interfaz significa comprometerse a cumplir un contrato. En el código entregado no se declara ninguna interface.

Ejemplo complementario: interfaz (NO implementado en NEXUS)

```
interface IInteractuable { void Interactuar(Cadete cadete); }
class Terminal : IInteractuable {
    public void Interactuar(Cadete cadete) {
        cadete.GanarExperiencia(5);
    }
}
```

Una clase abstracta puede incluir estado, constructor y métodos implementados; una interfaz define capacidades que diferentes clases pueden implementar sin compartir una clase base. Los métodos abstractos exigen implementación en clases derivadas concretas.

IV. Colecciones y LINQ

Uso real: Inventario declara private List<Objeto> objetos, la inicializa con new List<Objeto>(), agrega mediante Add, consulta Count, recorre con for y elimina con RemoveAt. Usar(numero, cadete) traduce el número humano (empieza en 1) a índice de lista (empieza en 0).

```
// Patrón real de Inventario:
objetos.Add(objeto);
for (int i = 0; i < objetos.Count; i++)
    objetos[i].Mostrar();
objetos.RemoveAt(numero - 1);
```

LINQ y expresiones lambda no aparecen en el código entregado. Los siguientes ejemplos son COMPLEMENTARIOS; requieren using System.Linq y una lista accesible.

```
using System.Linq;
List<Objeto> items = new List<Objeto> {
    new Objeto("Celula", "Energia", 25),
    new Objeto("Escudo", "Escudo", 20),
    new Objeto("Bateria", "Energia", 10)
};
var energia = items.Where(o => o.Tipo == "Energia");
var nombres = items.Select(o => o.Nombre);
var ordenados = items.OrderBy(o => o.Poder);
var fuertes = items.Where(o => o.Poder >= 20)
                   .OrderByDescending(o => o.Poder)
                   .Select(o => o.Nombre);
```

Where filtra, Select transforma y OrderBy ordena ascendentemente. En una lambda, o => o.Poder >= 20 significa: para cada objeto o, comprobar si su poder es al menos 20. Frente a un for tradicional, LINQ expresa la intención de la consulta de forma más compacta; el for ofrece control explícito de índices y pasos.

V. Análisis del funcionamiento

Flujograma: completar la misión

```
INICIO
  |
Crear SistemaNexus, Mision, Zonas y Cadete
  |
Mostrar menu y elegir Explorar
  |
Explorar Anomalia --> marcar llegada; gastar 10 energia
  |
Encuentro con Enemigo --> elegir accion
  |
¿Cadete con vida? -- NO --> no se marca nucleo/supervivencia
  | SI
Agregar nucleo; marcar nucleo y supervivencia; +30 XP
  |
Ir a Extraccion
  |
¿Llegada, nucleo, supervivencia y vida > 0?
  | SI                          | NO
Marcar regreso             Extraccion denegada
  |
Mision.EstaCompletada() --> premio y fin de mision
```

Seguimiento de estado verificable

Al crear Cadete("Alex"): Vida=100, Energia=60, Experiencia=0. Al explorar el bosque una vez: Vida=100, Energia=55, Experiencia=10; además se añade una célula de energía al inventario. Si después usa esa célula (+25), Energia pasa de 55 a 80 y el objeto usado se elimina. El inventario inicial también contiene una célula, escudo y analizador.

Predicción de ejecución

Si se entra a Ruinas y se elige Atacar, Robot comienza con Vida=50. Atacar aplica 50 puntos de daño: Robot queda con Vida=0, se considera derrotado y el cadete gana 25 XP. No se ejecuta Robot.Actuar porque el robot ya fue derrotado.

¿Qué pasa si modificamos el código?

Si el daño de Atacar cambia de 50 a 30, un Robot con 50 de vida queda con 20, no se entrega el premio de 25 XP y se ejecuta Robot.Actuar, que quita 10 de vida y 5 de energía al cadete. Este cambio afecta el resultado del encuentro y la supervivencia.

Observaciones de diseño importantes

El método Escapar recibe personaje pero no lo utiliza. El método Usar de Inventario imprime que el escudo se activa, pero no aumenta una propiedad de defensa ni reduce daños posteriores. Encuentro permite una sola acción por visita; si el enemigo sigue vivo, no hay bucle de combate dentro del mismo encuentro. La misión puede marcar núcleo y supervivencia tras escapar del enemigo, siempre que el cadete siga con vida. Estas observaciones se derivan de las condiciones y llamadas actuales.

VI. Preparación para el examen: seis preguntas razonadas

1. POO: ¿por qué Mision es una clase y no cuatro variables sueltas?

Respuesta razonada: Porque agrupa cuatro estados relacionados y las operaciones que los cambian. EstaCompletada centraliza la condición de finalización, evitando duplicarla en todo el sistema.

2. Clases: ¿cuál es el estado inicial de new Cadete("Ana") y qué hace base?

Respuesta razonada: Nombre="Ana", Vida=100, Energia=60, Experiencia=0 e Inventario nuevo. base(nombre,100) ejecuta el constructor de Personaje y establece Nombre y Vida.

3. Polimorfismo: ¿qué sucede al llamar Actuar desde un Personaje que apunta a Robot?

Respuesta razonada: Se ejecuta Robot.Actuar por despacho dinámico: imprime sus mensajes y aplica al cadete 10 puntos de daño y 5 de gasto energético.

4. Interfaces: ¿por qué usar IInteractuable para terminales y puertas?

Respuesta razonada: Es un ejemplo complementario: clases distintas pueden prometer Interactuar sin heredar de una misma clase concreta. El código que las utiliza depende del contrato, no de su implementación.

5. LINQ: con poderes 25, 20 y 10, ¿qué devuelve Where(o => o.Poder >= 20).Select(o => o.Nombre)?

Respuesta razonada: Devuelve los nombres "Celula" y "Escudo" en ese orden de origen. Where elimina Bateria (10) y Select convierte los objetos restantes en cadenas.

6. Predicción: el cadete tiene energía 60; explora Anomalia y escapa. ¿con cuánta energía queda y se completa el núcleo?

Respuesta razonada: Queda con 40: gasta 10 al entrar y 10 al escapar. Como sigue vivo, el método ExplorarAnomalia marca núcleo y supervivencia, aunque no haya derrotado al enemigo. Es una particularidad del código actual.

Repaso final y práctica individual

Antes del H3, poder explicar sin ayuda: qué hace cada clase; qué diferencias hay entre abstract, virtual y override; cómo cambia el estado del cadete; cómo se agregan y eliminan objetos; y cómo predecir el resultado de una condición o de una consulta LINQ. Practicar escribiendo un constructor sobrecargado, una interfaz y dos consultas LINQ en un archivo separado de pruebas.

Diagnóstico del PROMPT ESPEJO — pendiente de realización

La consigna exige incorporar íntegramente el resultado del PROMPT ESPEJO al final del documento. Todavía no se ha ejecutado el diagnóstico final del proceso de aprendizaje; por eso no se inventa una evaluación personal. Una vez terminado el estudio, enviar exactamente el prompt proporcionado por el docente en el mismo chat y anexar la respuesta completa aquí.

Referencia del código: archivo proporcionado por el estudiante con namespace NEXUS, clases Program, SistemaNexus, Personaje, Cadete, Robot, Enemigo, Objeto, Inventario, Zona y Mision.